
Software Design and Architecture

*By Habtam D.
AASTU, 2022*

Chapter Five

Documenting and Evaluating Software Architecture

Documenting Software Architecture

- Architecture serves as the **blueprint** for the system, and the project that develops it.
 - » It defines the **work assignments**.
 - » It is the **primary carrier of quality attributes**.
 - » It is the **best artifact** for **early analysis**.
 - » It is the key to **post-deployment maintenance**.
 - » It is the **conceptual glue** that **holds every phase of the project together** for all of its many stakeholders.
- It must be described in **sufficient detail, without ambiguity, and organized** so that all stakeholders can quickly find the information they need.
- Documentation speaks for the architect, **today** and **20 years** from today.

Cont'd

- Even the best architecture will be useless if the people who need it,
 - » do not know what it is;
 - » cannot understand it well enough to use, build, or modify it;
 - » misunderstand it and apply it incorrectly.
- All of the effort, analysis, hard work, and insightful design on the part of the architecture team will have been wasted.
- Architecture documentation must
 - » be sufficiently **transparent and accessible** to be quickly understood by new employees.
 - » be sufficiently concrete to serve as a **blueprint** for construction.
 - » have **enough information** to serve as a basis for analysis.

Cont'd

- Architecture documentation is both ***prescriptive*** and ***descriptive***.
 - » For some audiences, it prescribes ***what should be true***, placing constraints on decisions yet to be made.
 - » For other audiences, it describes ***what is true***, recounting decisions already made about a system's design.
- Understanding stakeholder uses of architecture documentation is essential.

Stakeholders Who Use the Architectural Documentation

- Architect and requirements engineers who represent the customer(s)
- Architect and designers of constituent parts
- Implementers
- Testers and integrators
- Maintainers
- Designers of other systems with which this one must interoperate
- Quality attribute specialists
- Managers
- Product line managers
- Quality assurance team
- etc.

Three Uses for Architecture Documentation

● Education

- » Introducing people to the system
 - New members of the team
 - External analysts or evaluators
 - New architect

● Primary vehicle for communication among stakeholders

- » Especially architect to developers
- » Especially architect to future architect!

● Basis for system analysis and construction

- » Architecture tells implementers what to implement.
- » Each module has interfaces that must be provided and uses interfaces from other modules.
- » Documentation can serve as a receptacle for registering and communicating unresolved issues.
- » Architecture documentation serves as the basis for architecture evaluation.

Notations

- *Informal notations*
 - » Views are depicted (often graphically) using **general-purpose diagramming** and **editing tools**.
 - » The semantics of the description are characterized in **natural language**.
 - » They cannot be formally analyzed.
- *Semiformal notations*
 - » **Standardized notation** that prescribes **graphical elements** and **rules of construction**.
 - » **Lacks a complete semantic treatment** of the meaning of those elements.
 - » Rudimentary analysis can be applied.
 - » UML is a semiformal notation in this sense.

Cont'd

- *Formal notations*
 - » Views are described in **a notation** that **has a precise** (usually mathematically based) **semantics**.
 - » Formal analysis of both syntax and semantics is possible.
 - » Architecture description languages (ADLs).
 - » Support automation through associated tools.

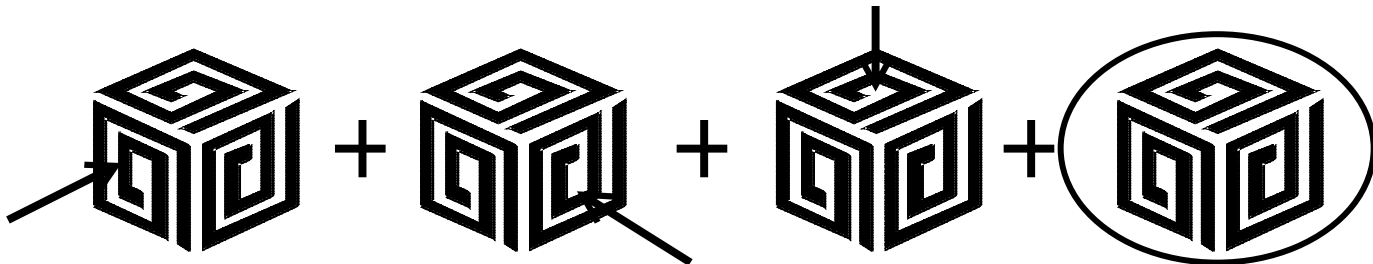
Choosing Notations

- Tradeoffs
 - » Typically, more **formal notations** take more **time and effort** to create and understand, but **offer reduced ambiguity and more opportunities** for analysis.
 - » Conversely, more **informal notations** are **easier** to create, but they **provide fewer guarantees**.
- Different notations are better (or worse) for expressing different kinds of information.
 - » UML class diagram will not help you reason about schedulability, nor will a sequence chart tell you very much about the system's likelihood of being delivered on time.
 - » Choose your notations and representation languages knowing the **important issues you need to capture** and reason about.

“Views and Beyond” Approach to Documentation

- Views let us divide a software architecture into a number of interesting and manageable representations of the system.
- Views give us our first principle of architecture documentation:

*Document the relevant views,
and then add information
that applies to more than one view.*



Which Views are Relevant?

- **Kruchten's 4+1 views**

- » **Logical view:** supports behavioral requirements. Key abstractions, which are objects or object classes
- » **Process view:** addresses concurrency and distribution. Maps threads to objects.
- » **Development view:** organization of software modules, libraries, subsystems, units of development.
- » **Physical view:** maps other elements onto processing and communication nodes.
- » **“Plus one” view:** Maps the other views onto important use cases to show how they work.

Cont'd

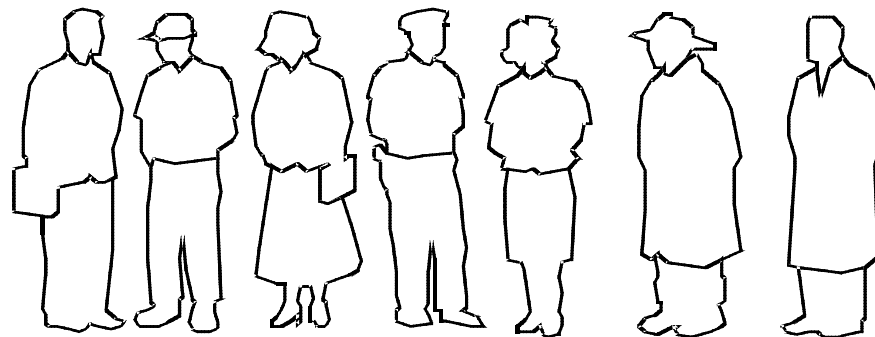
- **Siemens Four-Views (Hofmeister, Nord, Soni, *Applied Software Architecture*, 2000):**
 - » Conceptual view
 - » Module interconnection view
 - » Execution view
 - » Code view
- **Herzum & Sims (*Business Component Factory*, 1999):**
 - » Technical architecture
 - » Application architecture
 - » Project management architecture
 - » Functional architecture

Cont'd

- **Software Cost Reduction method (Parnas, et al., 1980s)**
 - » **Module view:** shows modules as units of encapsulation; used to isolate changes and achieve modifiability.
 - » **Process view:** shows processes and how they synchronize and communicate at run-time; used to achieve performance.
 - » **Uses view:** shows programs and how they depend on each other; used to achieve incremental development and the ability to quickly field subsets.

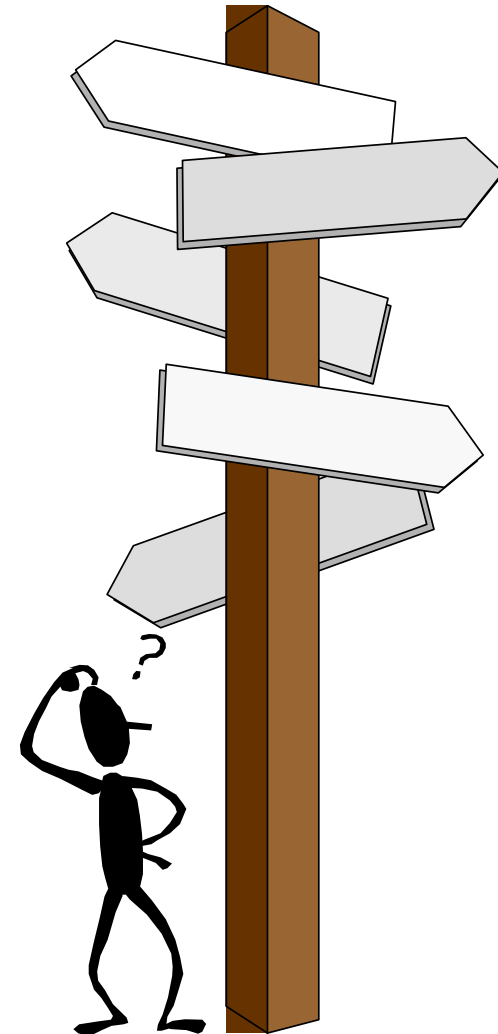
Cont'd

- Which views are relevant? It depends on:
 - » who the stakeholders are
 - » how they will use the documentation.
- Three primary uses for architecture documentation:
 - » Education -- introducing people to the project
 - » Communication -- among stakeholders
 - » Analysis -- assuring quality attributes



What views are available?

- Plenty! Too many!
- Already, we've seen 15 different views, and many more are available, based on examining the literature.
- An architect needs a way to choose the useful ones.
- One thing that would help is to organize the views into broad categories.

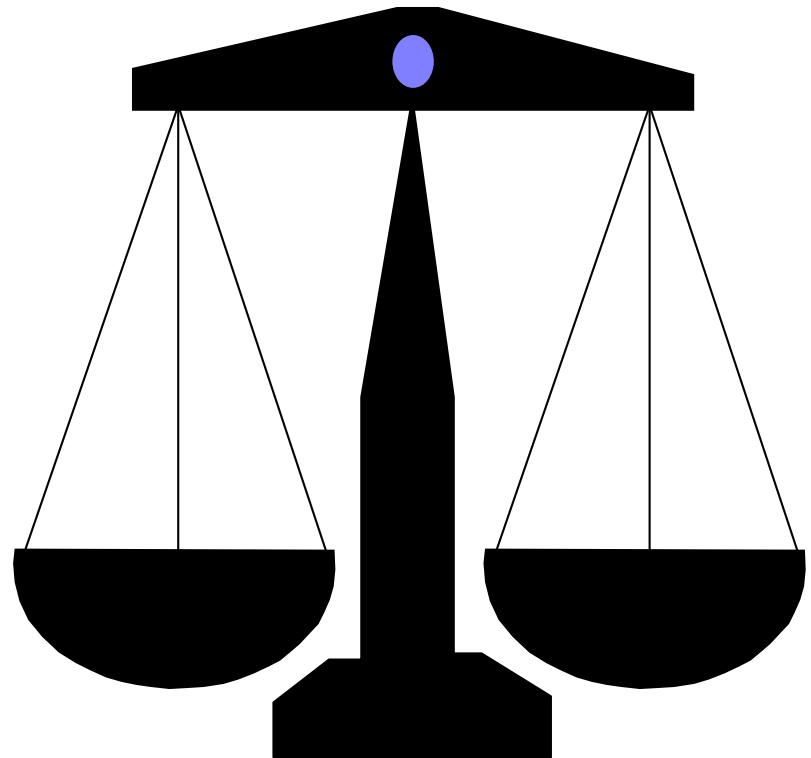


Cont'd

- An architect must consider the system in **three** ways:
 - » How is it structured as a set of code units?
 - e.g Module views (module *viewtype*)
 - » How is it structured as a set of elements that have run-time behavior and interactions?
 - e.g Component-and-connector views (C&C *viewtype*)
 - » How does it relate to non-software structures in its environment?
 - e.g Allocation views (allocation *viewtype*)

Cont'd

- ***How many views*** do we need in our documentation package?
 - » Each view comes with a ***cost and benefit***.
 - » ***Planning a view set*** requires understanding the ***needs of the stakeholders***, and the ***resources available***.



Choosing the Views

- You can determine ***which views are required, when to create them, and how much detail to include*** if you know the following:
 - » What people, and with what skills, are available
 - » Which standards you have to comply with
 - » What budget is on hand
 - » What the schedule is
 - » What the information needs of the important stakeholders are
 - » What the driving quality attribute requirements are
 - » What the approximate size of the system is?

Cont'd

- At a minimum, expect to have
 - » at least one module view,
 - » at least one C&C view, and
 - » for larger systems, at least one allocation view in your architecture document.

Method for Choosing the Views

- **Step 1. Build a stakeholder/view table.**
 - » **Rows:** *List the stakeholders* for your project's software architecture documentation
 - » **Columns:** Enumerate the *views* that apply to your system.
 - Use the structures discussed in Chapter 1, the views discussed in this chapter, and the views that your design work in ADD has suggested as a starting list of candidates.
 - Include the views or view sketches you have as a result of your design work so far.
 - » Some views (such as decomposition, uses, and work assignment) apply to every system, while others (various C&C views, the layered view) only apply to some systems.
 - » Fill in each cell to describe how much information the stakeholder requires from the view: none, overview only, moderate detail, or high detail.

Cont'd

- **Step 2. Combine views to reduce their number**
 - » Look for marginal views in the table; those that ***require only an overview, or that serve very few stakeholders.***
 - » Combine each marginal view with another view that has a stronger constituency.
 - » These views often combine naturally:
 - ***Various C&C views*** - Because C&C views all show runtime relations among components and connectors of various types, they tend to combine well.
 - ***Deployment view with either SOA or communicating-processes views***
 - An SOA view shows services, and a communicating-processes view shows processes. In both cases, these are components that are deployed onto processors.
 - ***Decomposition view and any of work assignment, implementation, uses, or layered views*** - The decomposed modules form the units of work, development, and uses. In addition, these modules populate layers.

Cont'd

- **Step 3. Prioritize and stage.**
 - » The decomposition view (one of the module views) is a particularly helpful view to release early.
 - High-level decompositions are often easy to design
 - The project manager can start to staff development teams, put training in place, determine which parts to outsource, and start producing budgets and schedules.
 - » You don't have to satisfy all the information needs of all the stakeholders to the fullest extent.
 - Providing 80 percent of the information goes a long way, and this might be “good enough” so that the stakeholders can do their job.
 - Check with the stakeholder if a subset of information would be sufficient.
 - » You don't have to complete one view before starting another.
 - People can make progress with overview-level information
 - A breadth-first approach is often the best.

Building the Documentation Package

- Documentation package consists of
 - » Views
 - » Documentation beyond views

Documenting a View

- **Section 1: The Primary Presentation.**
 - » The **primary presentation** shows the **elements and relations** of the view.
 - » The primary presentation should contain the information you wish to **convey about the system**—in the vocabulary of that view.
 - » The primary presentation is **most often graphical**.
 - It might be a diagram you've drawn in an informal notation using a simple drawing tool, or it might be a diagram in a semiformal or formal notation imported from a design or modeling tool that you're using.
 - If your primary presentation is graphical, make sure to include **a key that explains the notation**.
 - Lack of a key is the most common mistake that we see in documentation in practice.
 - » Occasionally the primary presentation will be **textual**, such as a **table or a list**.
 - If that text is presented according to certain stylistic rules, these rules should be **stated or incorporated by reference**, as the analog to the graphical notation key.

Cont'd

● Section 2: The Element Catalog.

- » The ***element catalog*** details at least those elements depicted in the primary presentation.
 - For instance, if a diagram shows elements A, B, and C, then the element catalog needs to explain what A, B, and C are.
 - If elements or relations relevant to this view were omitted from the primary presentation, they should be introduced and explained in the catalog.
- » Parts of the catalog:
 - ***Elements and their properties-*** This section names each element in the view and lists the properties of that element. Each view introduced in Chapter 1 listed a set of suggested properties associated with that view.
 - ***Relations and their properties-*** Each view has specific relation types that it depicts among the elements in that view.
 - ***Element interfaces-*** This section documents element interfaces.
 - ***Element behavior-*** This section documents element behavior that is not obvious from the primary presentation.

Cont'd

- **Section 3: Context Diagram.**

- » A **context diagram** shows *how the system or portion of the system depicted* in this view relates to *its environment*.
- » The purpose of a context diagram is *to depict the scope of a view*.
- » Entities in the environment may be humans, other computer systems, or physical objects, such as sensors or controlled devices.

- **Section 4: Variability Guide.**

- » A **variability guide** shows *how to exercise any variation points* that are a part of the architecture shown in this view.

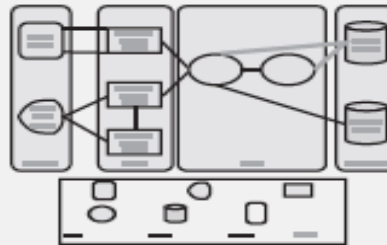
- **Section 5: Rationale.**

- » **Rationale** explains *why the design reflected in the view came to be*.
- » The goal of this section is to explain *why the design is as it is and to provide a convincing argument that it is sound*.
- » The **choice of a pattern** in this view should be justified here by describing the architectural problem that the chosen pattern solves and the rationale for choosing it over another.

View Template

Template for a View

Section 1. Primary Presentation



Section 2. Element Catalog

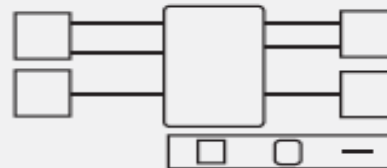
Section 2.A. Elements and Their Properties

Section 2.B. Relations and Their Properties

Section 2.C. Element Interfaces

Section 2.D. Element Behavior

Section 3. Context Diagram



Section 4. Variability Guide

Section 5. Rationale

Documenting Information Beyond Views

- **Document control information.**
 - » List the *issuing organization, the current version number, date of issue and status, a change history, and the procedure for submitting change requests* to the document.
 - » Usually captured in the *front matter*.

Cont'd

- **Section 1: Documentation Roadmap.** The documentation map tells the reader *what information is in the documentation and where to find it*.
 - » **Scope and summary** - Explain the purpose of the document and briefly summarize what is covered.
 - » **How the documentation is organized** - For each section in the documentation, give a short synopsis of the information that can be found there.
 - » **View overview** - Describes the views that the architect has included in the package. For each view::
 - The name of the view and what pattern it instantiates, if any.
 - A description of the view's element types, relation types, and property types.
 - A description of language, modeling techniques, or analytical methods used in constructing the view.
 - » **How stakeholders can use the documentation:**
 - This section shows how various stakeholders might use the documentation to help address their concerns.
 - Include short scenarios, such as “A maintainer wishes to know the units of software that are likely to be changed by a proposed modification.”
 - To be compliant with ISO/IEC 42010-2007, you must consider the concerns of at least users, acquirers, developers, and maintainers.

Cont'd

- **Section 2: How a View is Documented.**
 - » Explain the *standard organization you're using to document views*—either the one described in this chapter or one of your own.
- **Section 3: System Overview.**
 - » Short prose *description of the system's function, its users, and any important background or constraints.*
 - » Provides your readers with a consistent mental model of the system and its purpose.
 - » This might be a pointer to your project's concept-of-operations document for the system.

Cont'd

- **Section 4: Mapping Between Views.**
 - » *Helping a reader understand the associations between views* will help that *reader gain a powerful insight into how the architecture works* as a unified conceptual whole.
 - » The associations between elements across views in an architecture are, in general, many-to-many.
 - » View-to-view associations can be captured as tables.
 - The **table** should name the correspondence between the elements across the two views.
 - **Examples**
 - “is implemented by” for mapping from a component-and-connector view to a module view
 - “implements” for mapping from a module view to a component-and-connector view
 - “included in” for mapping from a decomposition view to a layered view

Cont'd

- **Section 5: Rationale.**

- » Documents the ***architectural decisions*** that apply to more than one view.
 - Documentation of background or organizational constraints or major requirements that led to decisions of system-wide import.
 - Decisions about which fundamental architecture patterns are used.

- **Section 6: Directory.**

- » Set of ***reference material that helps readers*** find more information quickly.
 - Index of terms
 - Glossary
 - Acronym list.

Documenting Behavior

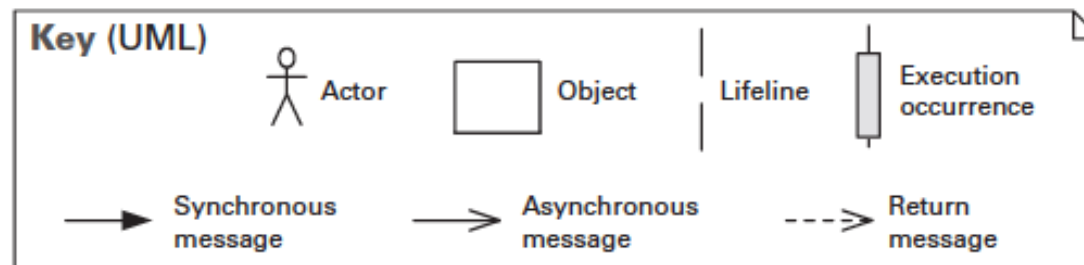
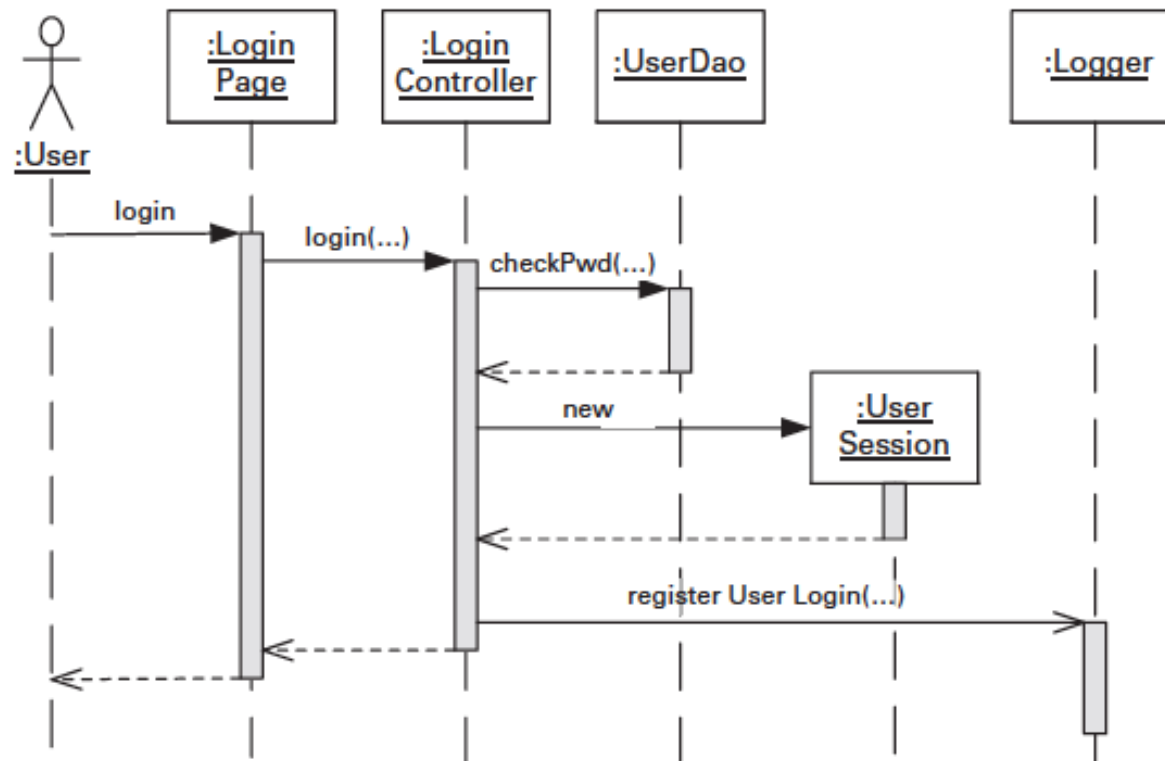
- Behavior documentation complements each views by describing ***how architecture elements in that view*** interact with each other.
- Behavior documentation enables reasoning about
 - » a system's potential to deadlock
 - » a system's ability to complete a task in the desired amount of time
 - » maximum memory consumption
 - » and more
- Behavior has its own section in our view template's element catalog.

Notations for Documenting Behavior

- ***Trace-oriented languages***

- » ***Traces*** are sequences of activities or interactions that describe the ***system's response*** to a specific stimulus when the system is in a specific state.
- » A ***trace*** describes a ***particular sequence of activities or interactions*** between ***structural elements*** of the system.
- » Examples
 - use cases
 - sequence diagrams
 - communication diagrams
 - activity diagrams
 - message sequence charts
 - timing diagrams
 - Business Process Execution Language

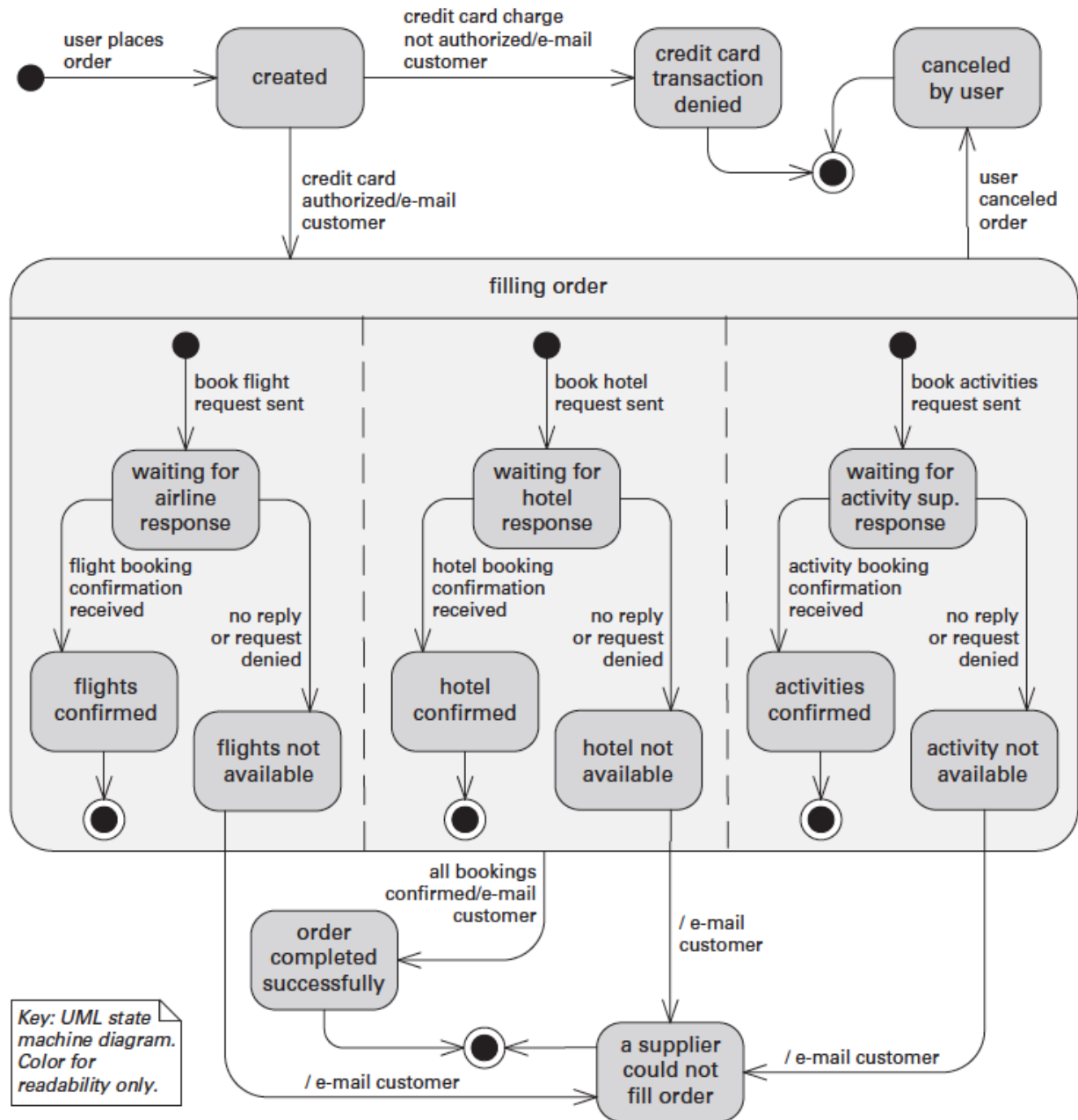
Sequence Diagram



Cont'd

- **Comprehensive languages**
 - » **Comprehensive models** show the **complete behavior** of structural elements.
 - » Given this type of documentation, it is possible **to infer all possible paths** from **initial state** to **final state**.
 - » The **state machine** formalism represents the behavior of architecture elements because each state is an abstraction of all possible histories that could lead to that state.
 - » **State machine languages** allow you to complement a structural description of the elements of the system with constraints on interactions and timed reactions to both internal and environmental stimuli.

State Machine



Documenting Quality Attributes

- Where do quality attributes show up in the documentation? There are five major ways:
 - » **Rationale** that explains the **choice of design approach** should include a discussion about the quality attribute requirements and tradeoffs.
 - » **Architectural elements** providing a service often have quality attribute bounds **assigned to them**, defined in the interface documentation for the elements, or recorded as *properties* that the elements exhibit.
 - » Quality attributes often **impart a “language” of things** that you would look for. Someone fluent in the “language” of a quality attribute can search for the kinds of architectural elements) put in place to satisfy that quality attribute requirement.
 - » Architecture documentation often contains a **mapping to requirements** that shows how requirements (including quality attribute requirements) are satisfied.
 - » Every quality attribute requirement will have a **constituency of stakeholders who want to know that it is going to be satisfied**. For these stakeholders, the **roadmap** tells the stakeholder where in the document to find it.

Documenting Architectures That Change Faster Than You Can Document Them

- An architecture that changes ***at runtime***, or ***as a result of a high-frequency release-and-deploy cycle***, change much faster than the documentation cycle.
- Nobody will wait until a new architecture document is produced, reviewed, and released.
- In this case:
 - » ***Document what is true about all versions of your system*** - Record those invariants as you would for any architecture. This may make your documented architecture more a description of constraints or guidelines that any compliant version of the system must follow.
 - » ***Document the ways the architecture is allowed to change*** - This will usually mean adding new components and replacing components with new implementations. The place to do this is called the variability guide.

Architecture Documentation Templates

- **Rational Unified Process / Kruchten 4+1**
- Siemens Four Views
- C4ISR architecture framework
- **IEEE Standard 1471 for architecture documentation**
- Hewlett Packard's architecture documentation framework
- Data flow and control flow views
- Enterprise architecture methods
- Zachman's Information Architecture
- RM-ODP
- Etc.

Seven Rules for Sound Documentation

(By P.Clements, et al.)

- **Rule 1:** Write Documentation from the Reader's Point of View
- **Rule 2:** Avoid Unnecessary Repetition
- **Rule 3:** Avoid Ambiguity
 - » **Rule 3a:** Explain Your Notation
- **Rule 4:** Use a Standard Organization (template)
- **Rule 5:** Record Rationale
- **Rule 6:** Keep Documentation Current but Not Too Current
- **Rule 7:** Review Documentation for Fitness of Purpose

Project Work– Software Architecture Documentation

Due Date: Any time before Final Exam starts

Use RUP - Architecture Documentation Template

Title Page

Change History

Table of Contents

List of Figures

1. Scope
2. References
3. Software Architecture
4. Architectural Goals & Constraints
5. Logical Architecture
6. Process Architecture
7. Development Architecture
8. Physical Architecture
9. Scenarios
10. Size and Performance
11. Quality

Appendices

A. Acronyms and Abbreviations

B. Definitions

C. Design Principles

Evaluating Software Architecture

- **Software architecture evaluation** is a method which determines the ***properties, strengths and weaknesses*** of software architecture.
- Software architectural evaluation provides assurance to developers that ***their chosen architecture will meet both functional and non-functional quality requirements.***
- It is to ***verify if the architectural decisions are appropriate*** in early stages and ***predict*** if a system will have the ***required quality attributes.***

Cont'd

- If we predict the ***impact a design decision*** has, and we ***choose the best option*** for our project, we will ***reduce the budget*** and obtain the ***software architecture we need***.
- The goal is to ***determine the degree in which a software architecture or an architectural style satisfy the quality requirements***.

When Can an Architecture Be Evaluated?

- As early as possible, even during actual development of SA.
 - » The earlier problems are found, the **earlier** and **cheaper** they can be fixed.
- Certainly after the architecture is completed, you should validate it before development.
- Later to ensure consistency between **design** and **implementation** especially for **legacy systems**.
- Before acquiring a new system.
- The real answer is **early** and **often**!

Who's Involved?

- ***Evaluation team*** - these are the people who will conduct the evaluation and perform the analysis.
- ***Stakeholders*** - stakeholders are people who have a vested interest in the architecture and the system.

Why To Evaluate an Architecture?

- Forces Preparation for Review
 - » ***Documentation/specifications*** must be provided, hence they must exist or be created.
 - » Some evaluations use ***standard questions***, and the ***architect can prepare ahead*** to ensure that the architecture scores well.
 - » Evaluations ***make the criteria*** for evaluation explicit by prioritizing requirements or quality goals.

Cont'd

- Early Detection of Problems

- » The ***problems that can be found*** by an ***architecture evaluation*** include:
 - unreasonable requirements.
 - performance problems.
 - problems associated with potential future modifications.
- » The ***earlier*** in the life cycle that ***problems are found***, the ***easier*** it is to fix them.

Cont'd

- Validation of Requirements
 - » Evaluations ***put stakeholders in the same room*** with each other, often for the ***first time***.
 - uncovers ***conflicts*** and ***tradeoffs***.
 - provides a forum for ***negotiated resolution*** of problems.
 - » It often results in ***the generation of new requirements*** or the ***clarification of existing requirements***.

Cont'd

- Improved Architectures
 - » Development organizations anticipate ***types of questions raised*** at evaluations and
 - design architectures with ***questions in mind***.
 - prepare ***documentation of the type needed*** at evaluation.
 - give ***explicit consideration to qualities*** to be evaluated.

Planned or Unplanned Evaluations

- Planned
 - » Normal part of software life cycle.
 - » Built into the project's work plans, budget and schedule.
 - » Scheduled right after completion of SA.
 - » Planned evaluations are “proactive” and “team-building”.
- Unplanned
 - » As a result of some serious problem.
 - » “mid-course correction”.
 - » Challenge to technical authority of Team.
 - » Unplanned evaluations are “reactive” and “tension-filled”.

Software Architecture Evaluation Methods

- In the last years, several methods for evaluating software architectures have been defined
 - » **Scenario-based Software Architecture Evaluation Methods**
 - SAAM, ATAM, ALPSM and ALMA, SBAR, SALUTA, SAAMCS, ESAAMI, etc.
 - » **Mathematical Model-based Software Architecture Evaluation**
 - Software Architecture-based Reliability Analysis (e.g., Cheung [30])
 - Software Architecture-based Performance Analysis (e.g., BIM, CM, ABI)
 - » **Metrics-based Software Architecture Evaluation Methods**
 - **Early evaluation:** QuADAI
 - **Late evaluation:** TLC, FA, MNS, SSC

Cont'd

- After extensive research, the Carnegie Mellon **Software Engineering Institute (SEI)** has come up with three methods:
 - » Architecture Tradeoff Analysis Method (ATAM)
 - » Software Architecture Analysis Method (SAAM)
 - » Architecture Review For Intermediate Design (ARID)
- In many expert opinions, **ATAM** is the best method of the three.

Reading Assignment

- *Architecture Tradeoff Analysis Method (ATAM)* (*Reference Book -Software Architecture In Practice - Chapter 11*)
 - » *Participants in ATAM*
 - » *Phases in ATAM*
 - » *Conceptual flow of ATAM*
 - » *etc.*
- *For What Qualities Can We Evaluate an Architecture?*